Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 4

Naïve Bayes, Text Classification & Sentiment Analysis

The Task of Text Classification

Classification

- Classification lies at the heart of both human and machine intelligence.
  - Deciding what letter, word, or image has been presented to our senses
  - recognizing faces or voices
  - sorting mail
  - assigning grades to homeworks
- These are all examples of assigning a category to an input.

Classification: Is this spam?

Subject: **Important notice!**
From: Stanford University <newsforum@stanford.edu>
Date: October 28, 2011 12:34:16 PM PDT
To: undisclosed-recipients:;

Greats News!

You can now access the latest news by using the link below to login to Stanford University News Forum.

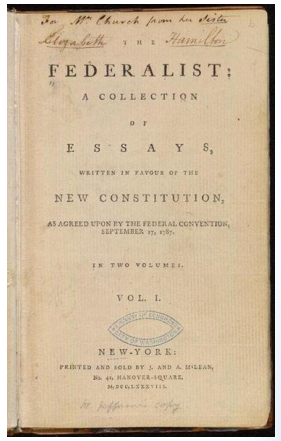
http://www.123contactform.com/contact-form-StanfordNew1-236335.html

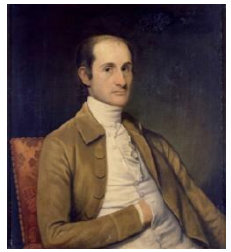
Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information
about the new services.

© Stanford University. All Rights Reserved.

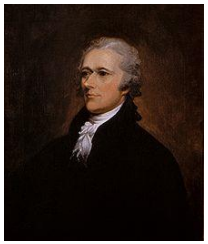
Who wrote which Federalist papers?

- 1787-1788: anonymous essays try to convince New York to ratify U.S Constitution: Alexander Hamilton, James Madison, and John Jay.
- Authorship of 12 of the letters in dispute
- 1963: solved by Mosteller and Wallace using Bayesian methods

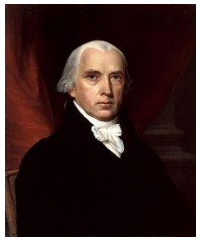




John Jay



Alexander Hamilton



James Madison

Positive or negative movie review?



- Unbelievably disappointing
- Full of zany characters and richly applied satire, and some great plot twists



- This is the greatest screwball comedy ever filmed



- It was pathetic. The worst part about it was the boxing scenes.



What is the subject of this article?

MEDLINE Article



?

MeSH (Medical Subject Headings)
Subject Category Hierarchy

- Antogonists and Inhibitors
- Blood Supply
- Chemistry
- Drug Therapy
- Embryology
- Epidemiology
- …

Text Classification

- Assigning subject categories, topics, or genres
- Spam detection
- Authorship identification
- Age/gender identification
- Language Identification
- Sentiment analysis
- …

Classification below document level

- Period disambiguation (period is EOS or part of a word)
- Word tokenization (deciding if a character should be a word boundary)
- Sentence tokenization or sentence type categorization
- Part-of-speech tagging
- Etc.

Text Classification: definition

- *Input*:
  - a document d
  - a fixed set of classes $C = \{c_1, c_2, \ldots, c_J\}$

- *Output*: a predicted class $c \in C$

Classification Methods: Hand-coded rules

- Rules based on combinations of words or other features
- Spam: black-list-address OR (expressions like “dollars” AND “have been selected”; or “Without any cost”; or “Dear Winner”)
- Accuracy can be high
  - If-rules carefully refined by expert
- But building and maintaining these rules is expensive

Classification Methods: Supervised Machine Learning

- *Input:*
  - A document *d*
  - A fixed set of classes $C = \{c_1, c_2, \ldots, c_J\}$
  - A training set of *m* hand-labeled documents $(d_1, c_1), \ldots, (d_m, c_m)$
- *Output:*
  - A learned classifier $\gamma: d \rightarrow c_i$

Classification Methods: Supervised Machine Learning

- Any kind of classifier
  - Naïve Bayes
  - Logistic regression
  - Support-vector machines
  - k-Nearest Neighbors
  - …

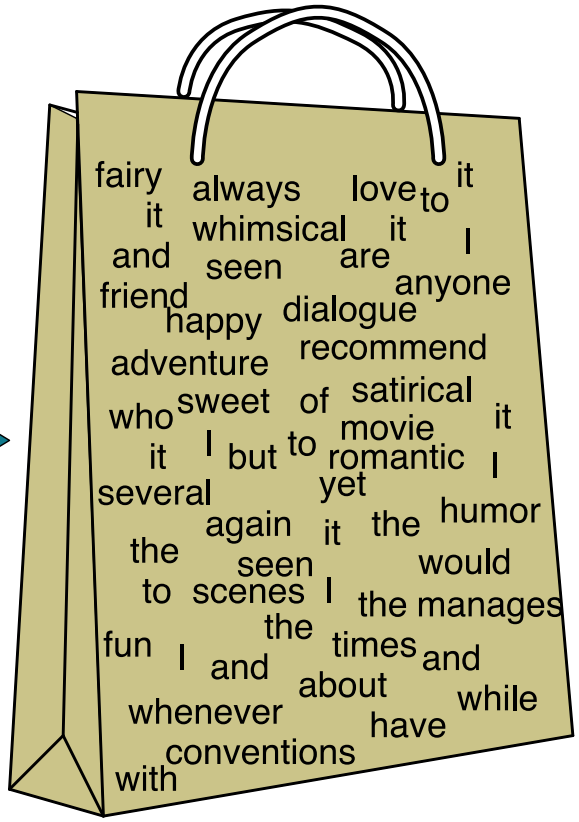
The Naive Bayes Classifier

Naive Bayes Intuition

- Simple ("naive") classification method based on Bayes rule
- Relies on a very simple representation of document
  - **Bag of words**

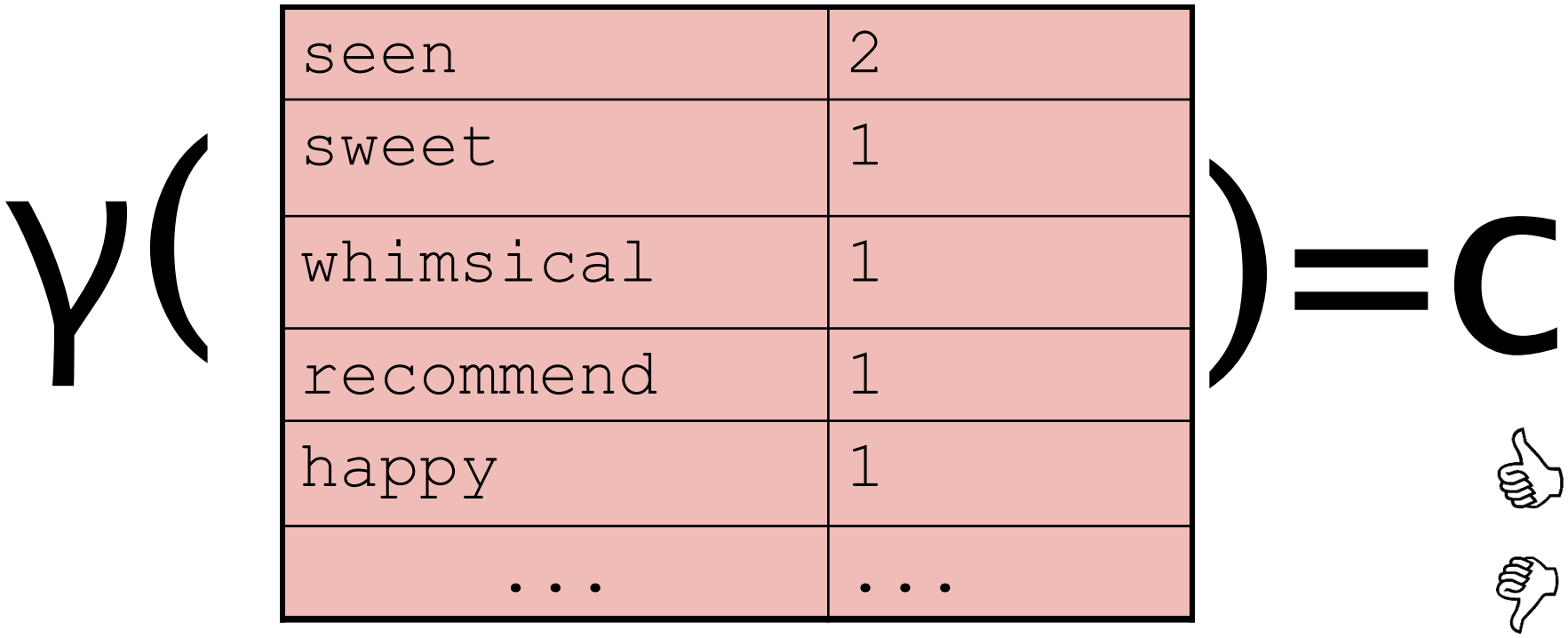
The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

The bag of words representation



Bayes' Rule Applied to Documents and Classes

- Bayesian inference has been known since the work of Bayes (1763)
- First applied to text classification by Mosteller and Wallace (1964)
- Bayesian classification intuition: use Bayes' rule to transform maximum a posteriori (MAP) $c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$ into other probabilities that have some useful properties

Naive Bayes Classifier

For a document *d* and a class *c*

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(c|d)$$

MAP is "maximum a posteriori" = most likely class

$$= \underset{c \in C}{\operatorname{argmax}} \frac{P(d|c)P(c)}{P(d)}$$

Bayes Rule

$$= \underset{c \in C}{\operatorname{argmax}} P(d|c)P(c)$$

Dropping the denominator

Naive Bayes Classifier

"Likelihood" "Prior"

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} \; P(d|c) \quad P(c)$$

$$= \underset{c \in C}{\operatorname{argmax}} \, P(x_1, x_2, ..., x_n \mid c) P(c)$$

Document d represented as features x1..xn

Generative NB

- **Naive Bayes** is said to be **a generative model** because the previous equation can be read as stating a kind of implicit assumption about how a document is generated:
  - First a class is sampled from P(c)
  - Then the words are generated by sampling from P(d|c).
- We could imagine generating artificial documents, or at least their word counts, by following this process.

Naïve Bayes Classifier

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(x_1, x_2, ..., x_n \mid c) P(c)$$

O(|X|^n•|C|) parameters

How often does this class occur?

Could only be estimated if a very, very large number of training examples was available. ➔ Too hard to compute

We can just count the relative frequencies in a corpus

Multinomial Naive Bayes Independence Assumptions

$$P(x_1, x_2, \ldots, x_n \mid c)$$

- **Bag of Words assumption**: Assume position doesn't matter (for text classification)
- **Conditional Independence**: Assume the feature probabilities $P(x_i \mid c_j)$ are independent given the class c_j.

$$P(x_1, \ldots, x_n \mid c) = P(x_1 \mid c) \bullet P(x_2 \mid c) \bullet P(x_3 \mid c) \bullet \ldots \bullet P(x_n \mid c)$$

Multinomial Naive Bayes Classifier

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(x_1, x_2, ..., x_n \mid c) P(c)$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{x \in X} P(x|c)$$

Applying Multinomial Naive Bayes Classifiers to Text Classification

- To apply the NB classifier to text:
  - Each word in the document is used as a feature
  - Each of the words in the document is considered by walking an index through every word position in the document

positions ← all word positions in test document

$$c_{NB} = \underset{c_j \in C}{\operatorname{argmax}} P(c_j) \prod_{i \in positions} P(x_i \mid c_j)$$

Problems with multiplying lots of probas

- As seen in the previous chapter, there's a problem with multiplying lots of probabilities :

$$c_{NB} = \underset{c_j \in C}{\operatorname{argmax}} P(c_j) \prod_{i \in positions} P(x_i \mid c_j)$$

- It can result in floating-point underflow!

.0006 * .0007 * .0009 * .01 * .5 * .000008….

- So use logs, because log(*ab*) = log(*a*) + log(*b*)
- Thus sum logs of probabilities instead of multiplying probabilities!

Work in log space

Instead of this:

$$c_{NB} = \underset{c_j \in C}{\operatorname{argmax}} P(c_j) \prod_{i \in positions} P(x_i \mid c_j)$$

This:

$$c_{\mathrm{NB}} = \underset{c_j \in C}{\operatorname{argmax}} \left[\log P(c_j) + \sum_{i \in \mathrm{positions}} \log P(x_i | c_j) \right]$$

Notes:

- Taking log doesn't change the ranking of classes!
- The class with highest probability also has highest log probability!

- It's a linear model:
  - Just a max of a sum of weights: a **linear** function of the inputs ➔ Naive Bayes is a **linear classifier**

Training the Naive Bayes Classifier

Learning the Multinomial Naive Bayes Model

- Maximum likelihood estimates
  - simply use the frequencies in the data

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}}$$

Class C_j prior, i.e. percentage of documents that are in class C_j; and N: total number of documents in the training set

$$\hat{P}(w_i \mid c_j) = \frac{count(w_i, c_j)}{\sum_{w \in V} count(w, c_j)}$$

Fraction of times the word w_i appears among all words in all documents of topic c_j

The vocabulary V consists of the union of all the word types in all classes

Parameter estimation

$$\hat{P}(w_i \mid c_j) = \frac{count(w_i, c_j)}{\sum_{w \in V} count(w, c_j)}$$

- Create a mega-document for topic *j* (class c_j) by concatenating all docs in this topic
  - Use frequency of w_i in the mega-document

Problem with Maximum Likelihood

- What if we have seen no training documents with the word ***fantastic*** and want to see if it is classified in the topic **positive** (*thumbs-up)*?

$$\hat{P}(\text{"fantastic"} \mid \text{positive}) = \frac{count(\text{"fantastic"}, \text{positive})}{\sum_{w \in V} count(w, \text{positive})} = 0$$

- Zero probabilities cannot be conditioned away, no matter the other evidence!

$$c_{MAP} = \text{argmax}_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

Laplace (add-1) smoothing for Naïve Bayes

$$\hat{P}(w_i \mid c) = \frac{count(w_i, c)}{\sum_{w \in V} \left(count(w, c) \right)} \qquad \hat{P}(w_i \mid c) = \frac{count(w_i, c) + 1}{\sum_{w \in V} \left(count(w, c) + 1 \right)}$$

$$= \frac{count(w_i, c) + 1}{\left(\sum_{w \in V} count(w, c) \right) + |V|}$$

While Laplace smoothing is usually replaced by more sophisticated smoothing algorithms, in language modeling, it is commonly used in Naive Bayes text categorization

Multinomial Naïve Bayes: Learning

- From the training corpus, extract *Vocabulary*

- Calculate the $P(c_j)$ terms

  For each c_j in C do

  $docs_j \leftarrow$ all docs with class = c_j

$$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$

- Calculate the $P(w_k \mid c_j)$ terms
  - $Text_j \leftarrow$ single doc containing all $docs_j$
  - For each word w_k in *Vocabulary*

    $n_k \leftarrow$ # of occurrences of w_k in $Text_j$

$$P(w_k \mid c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

$\alpha > 0$ ($\alpha = 1$: add-1 smoothing)

In this case, α is a hyper-parameter to be fine-tuned.

Unknown words

- What about unknown words
  - that appear in our test data
  - but not in our training data or vocabulary?
- We **ignore** them
  - Remove them from the test document!
  - Pretend they weren't there!
  - Don't include any probability for them at all!
- Why don't we build an unknown word model?
  - It doesn't help: knowing which class has more unknown words is not generally helpful!

Stop words

- Some systems ignore stop words
  - **Stop words:** very frequent words like *the* and *a*.
    - Sort the vocabulary by word frequency in training set
    - Call the top 10 to 100 most frequent words the **stopword list**.
    - Remove all stop words from both training and test sets as if they were never there!
- But removing stop words doesn't usually help
  - So in practice most NB algorithms use **all** words and **don't** use stopword lists

Sentiment and Binary Naive Bayes

Worked sentiment example!

- A number of sentences from movie reviews:

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Worked sentiment example with add-1 smoothing

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

1. Prior from training:

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}}$$

P(-) = 3/5

P(+) = 2/5

2. Drop "with"

3. Likelihoods from training:

$$p(w_i|c) = \frac{count(w_i, c) + 1}{(\sum_{w \in V} count(w, c)) + |V|}$$

$$P(\text{“predictable”}|-) = \frac{1+1}{14+20} \qquad P(\text{“predictable”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“no”}|-) = \frac{1+1}{14+20} \qquad P(\text{“no”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“fun”}|-) = \frac{0+1}{14+20} \qquad P(\text{“fun”}|+) = \frac{1+1}{9+20}$$

4. Scoring the test set:

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$

$$P(+)P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

Optimizing for sentiment analysis

- Standard naive Bayes text classification can work well for SA, but some small changes are generally employed that improve performance
- For tasks like SA, word **occurrence** seems to be more important than word **frequency**.
  - The occurrence of the word *fantastic* tells us a lot
  - The fact that it occurs 5 times may not tell us much more.
- This variant is called **Binary multinomial Naive Bayes**, or just **Binary NB**:
  - Clip the word counts at 1
  - Same algorithm as before except that
    - For each document remove all duplicate words before concatenating them into the single big document during training and
    - Remove duplicate words from test documents.

Binary Multinomial Naïve Bayes: Learning

- From the training corpus, extract *Vocabulary*

- Calculate the $P(c_j)$ terms

  For each c_j in C do

  $docs_j \leftarrow$ all docs with class $= c_j$

$$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$

- Calculate the $P(w_k \mid c_j)$ terms
  - Remove duplicates in each doc:
    - For each word type w in doc_j
      - Retain only a single instance of w
  - $Text_j \leftarrow$ single doc containing all $docs_j$
  - For each word w_k in *Vocabulary*

    $n_k \leftarrow$ # of occurrences of w_k in $Text_j$

$$P(w_k \mid c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

Binary Multinomial Naïve Bayes on a test document *d*

- First remove all duplicate words from *d*
- Then compute NB using the same equation:

$$c_{NB} = \underset{c_j \in C}{\operatorname{argmax}} P(c_j) \prod_{i \in positions} P(w_i \mid c_j)$$

Binary Multinomial Naïve Bayes

Example: 4 documents (shortened and text-normalized for this example) are remapped to binary, with the modified counts shown in the table.

Four original documents:

- − it was pathetic the worst part was the boxing scenes
- − no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- − it was pathetic the worst part boxing scenes
- − no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts		Binary Counts	
	+	−	+	−
and	2	0	1	0
boxing	0	1	0	1
film	1	0	1	0
great	3	1	2	1
it	0	1	0	1
no	0	1	0	1
or	0	1	0	1
part	0	1	0	1
pathetic	0	1	0	1
plot	1	1	1	1
satire	1	0	1	0
scenes	1	2	1	2
the	0	2	0	1
twists	1	1	1	1
was	0	2	0	1
worst	0	1	0	1

Counts can still be multiple in a class doc! Binarization is within-doc!

Sentiment Classification: Dealing with Negation

- I really like this movie

  I really **don't** like this movie

- Negation changes the meaning of "like" to negative.
- Negation can also change negative to (rather) positive
  - **Don't** dismiss this film
  - **Doesn't** let us get bored

Sentiment Classification: Handling Negation

Simple baseline method:

- Add NOT_ to every word after a token of logical negation (n't, not, no, never) until the next punctuation mark:

```
didn't like this movie, but I
```



```
didn't NOT_like NOT_this NOT_movie, but I
```

- Newly formed 'words' like NOT_like, NOT_recommend will thus occur more often in negative documents and act as cues for negative sentiment,
- Words like NOT_bored, NOT_dismiss will acquire positive associations.
- Syntactic parsing (Chapter 18) can be used to handle the details of the grammatical relationships
- This simple baseline works quite well in practice.

Sentiment Classification: Lexicons

- Sometimes not enough labeled training data to estimate +ve and -ve sentiment.
- In such cases one can instead derive the +ve and -ve word features from sentiment **lexicons**, i.e. pre-built word lists that are pre-annotated with +ve or -ve sentiment.
- Various publically available lexicons exist

MPQA Subjectivity Cues Lexicon

- MPQA (Multi-Perspective QA)
  - 6885 words from 8221 lemmas, annotated for intensity (strong/weak)
  - Lexicon contains
    - 6885 words (2718 positive, 4912 negative), along with
    - their POS-tagging,
    - labeled with polarity (positive, negative, neutral) and intensity (strong, weak).

+ : admirable, beautiful, confident, dazzling, ecstatic, favor, glee, great

- : awful, bad, bias, catastrophe, cheat, deny, envious, foul, harsh, hate

Using Lexicons in Sentiment Classification

- **Add a feature** that gets a count whenever a word from the lexicon occurs
  - E.g., a feature called "**this word occurs in the positive lexicon**" or "**this word occurs in the negative lexicon**"
- Now all positive words (*good, great, beautiful, wonderful*) or negative words count for that feature.
- **Worked out scenario.**
- Using 1-2 features isn't as good as using all the words.
- But when the training data is sparse or not representative of the test set, dense lexicon features can help
- Using just two features (positive and negative counts), the model is simplified while still capturing important sentiment information.
- ➔ This can help improve performance, especially in cases where individual words might not provide enough data due to sparsity.

Naive Bayes in Other tasks: Spam Filtering

- From the previous case we see that features in Naïve Bayes can express any property of the input text we want.
- **Spam detection**: A common solution, rather than using all the words as individual features, is to predefine likely sets of words or phrases as features, combined with features that are not purely linguistic.
- SpamAssassin Features (https://spamassassin.apache.org):
  - Mentions millions of (dollar) ((dollar) NN,NNN,NNN.NN) (regular expression)
  - From: starts with many numbers
  - Subject is all capitals
  - HTML has a low ratio of text to image area (not linguistic)
  - "One hundred percent guaranteed"
  - Claims you can be removed from the list

Naive Bayes in Language ID

- Determining what language a piece of text is written in:
  - Features based on character n-grams do very well
  - E.g. 2-grams ('zw') 3-grams ('nya', ' Vo'), or 4-grams ('ie z', 'thei'), or, even simpler byte n-grams
- Language ID systems are trained on multilingual text, such as Wikipedia (Wikipedia text in 68 different languages was used in [Lui and Baldwin, 2011]), or newswire
- Important to train on lots of varieties of each language (e.g., American English varieties like African-American English, or English varieties around the world like Indian English)

Summary: Naive Bayes is Not So Naive

- Very Fast, low storage requirements
- Works well with very small amounts of training data
- Robust to Irrelevant Features
- Very good in domains with many equally important features
- Optimal if the independence assumptions hold.
- A good dependable baseline for text classification
  - **But we will see other classifiers that give better accuracy**

Slide from Chris Manning

Naïve Bayes: Relationship to Language Modeling

Naïve Bayes and Language Modeling

- Naïve Bayes classifiers can use any sort of feature
  - URL, email address, dictionaries, network features
- But if, as in the previous slides,
  - We use **only** word features and
  - We use **all** of the words in the text (not a subset)
- Then
  - Naïve Bayes has an important similarity to language modeling.

Each class = a unigram language model

- A NB model can be viewed as a set of class-specific unigram language models.
- Assigning each word: P(word | c)
- Assigning each sentence: $P(s|c) = \prod_{i \in Positions} P(w_i|c)$

Class *pos*	
0.1	I
0.1	love
0.01	this
0.05	fun
0.1	film
…	

I	love	this	fun	film
0.1	0.1	0.01	0.05	0.1

P(s | pos) = 0.0000005

Naïve Bayes as a Language Model

- Which class assigns the higher probability to s?

Model pos	
0.1	I
0.1	love
0.01	this
0.05	fun
0.1	film

Model neg	
0.2	I
0.001	love
0.01	this
0.005	fun
0.1	film

I	love	this	fun	film
0.1	0.1	0.01	0.05	0.1
0.2	0.001	0.01	0.005	0.1

$P(s|pos) > P(s|neg)$

Note: this is just the likelihood part of the Naive Bayes model; once we multiply in the prior, a full NB model might well make a different classification decision.

Precision, Recall, and F1

Evaluating Classifiers: How well does our classifier work?

Let us first address binary classifiers:

- Is this email spam?

  spam (+) or not spam (-)

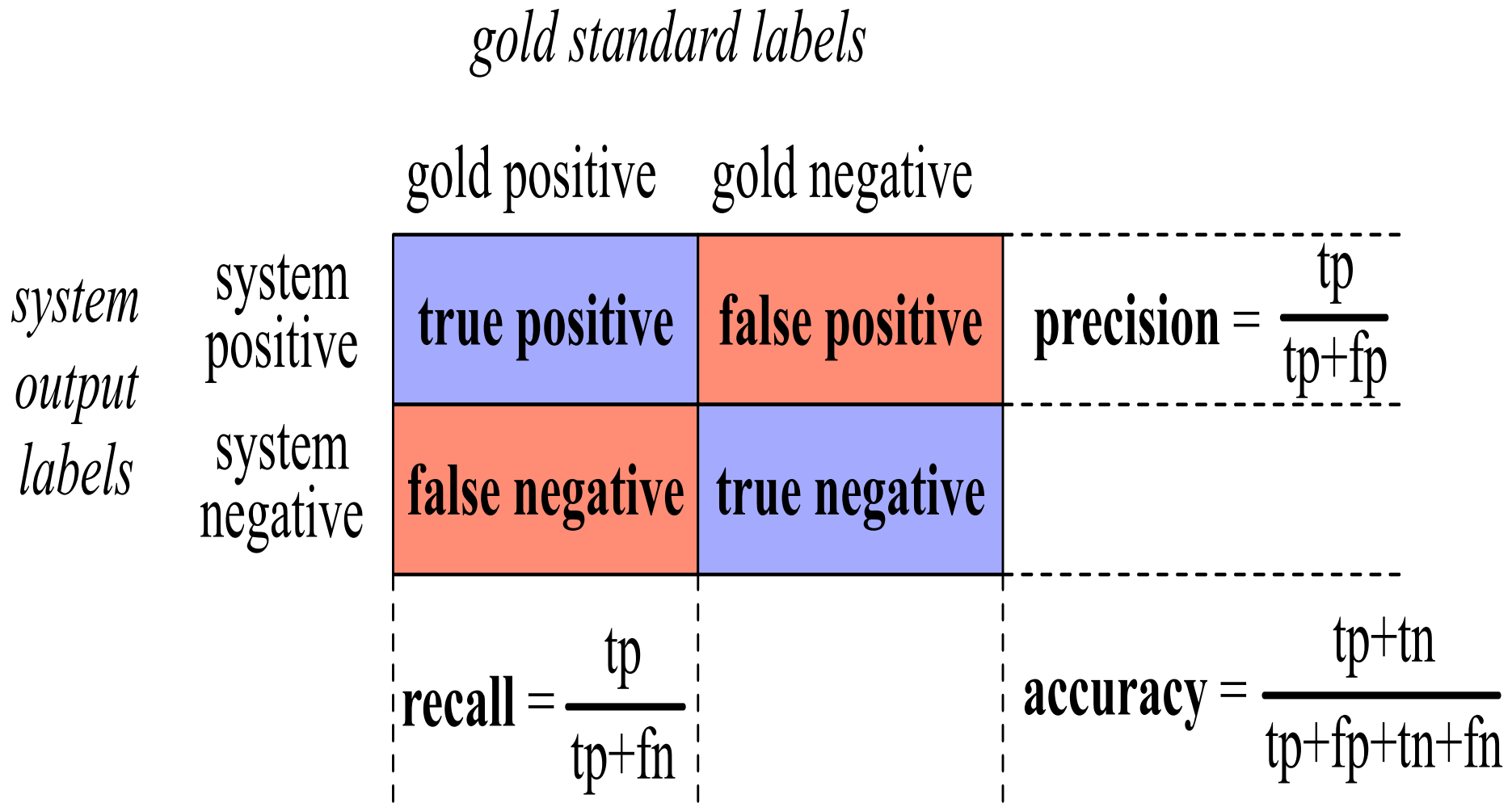
- Is this post about Delicious Pie Company?

  about Del. Pie Co (+) or not about Del. Pie Co(-)

We will need to know

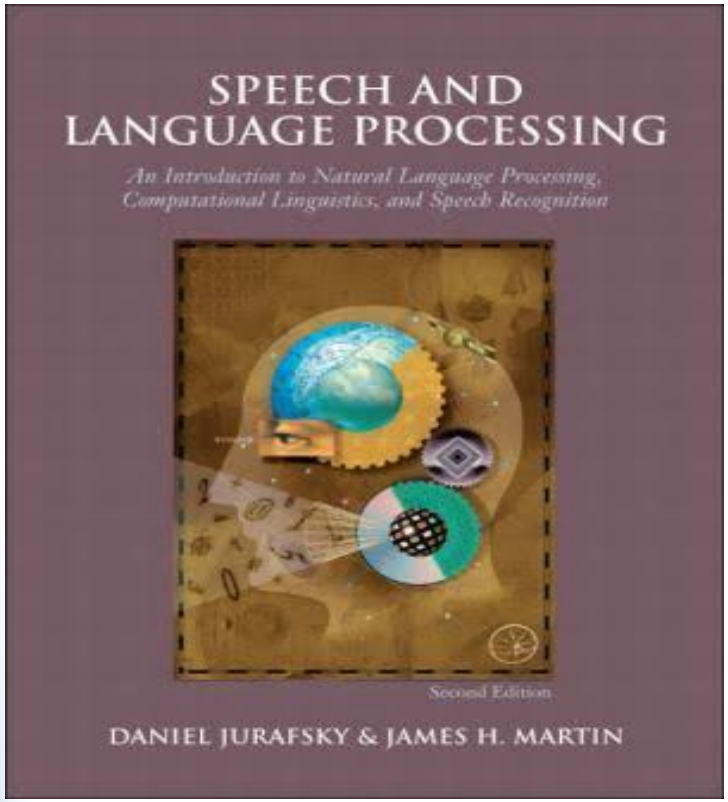
1. What did our classifier say about each email or post?
2. What should our classifier have said, i.e., the correct answer, usually as defined by humans ("gold label")

Confusion Matrix & Evaluation Metrics



Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.